



数字信号处理实验报告

实验内容: 实验一: 时域采样与频域采样

专 业: 工科试验班 (未来空天领军计划)

班 级: 242311 班

姓 名: 傅梁训

学 号: 24373088

任课教师: 白桐老师

2026 年 6 月

实验一：时域采样与频域采样

学 生： 傅梁训

指导老师： 白桐老师

摘 要

本实验围绕时域采样和频域采样两个基本理论展开。通过对指数衰减正弦模拟信号进行不同采样频率下的离散化和 FFT 分析，观察采样前后频谱的变化，理解采样频率不足时产生频谱混叠的原因。实验分别取 $F_s = 1000\text{ Hz}$ 、 300 Hz 和 200 Hz ，在 $T_p = 50\text{ ms}$ 的观测时间内生成采样序列，并统一采用 64 点 FFT 绘制幅频特性。

实验还对长度为 $M = 27$ 的有限长三角序列进行频域采样验证。通过对其频谱分别进行 32 点和 16 点等间隔采样，并作 IFFT，比较原序列、 $x_{32}(n)$ 和 $x_{16}(n)$ 的波形。结果表明，当频域采样点数 $N \geq M$ 时，时域不发生混叠；当 $N < M$ 时，IFFT 结果为原序列按 N 周期折叠后的主值区叠加序列。实验验证了“时域采样导致频域周期延拓，频域采样导致时域周期延拓”的对偶关系。

关键词：时域采样；频域采样；FFT；频谱混叠；周期延拓

Experiment 1: Time-domain Sampling and Frequency-domain Sampling

Author: Fu Liangxun

Tutor: Bai Tong

Abstract

This experiment studies two basic sampling theorems in digital signal processing. An exponentially decaying sinusoidal analog signal is sampled at different sampling rates, and its spectra are analyzed with a 64-point FFT. The results show that a higher sampling rate enlarges the spacing between periodic spectral replicas, while an insufficient sampling rate causes aliasing distortion.

The experiment also verifies the frequency-domain sampling theorem with a finite-length triangular sequence of length $M = 27$. The spectrum is sampled with 32 and 16 equally spaced frequency samples, followed by IFFT reconstruction. The 32-point result preserves the original sequence with zero padding, whereas the 16-point result becomes the periodic alias sum of the original sequence. The results confirm the duality between time-domain sampling and frequency-domain sampling.

Key words: time-domain sampling; frequency-domain sampling; FFT; aliasing; periodic extension

目 录

图目录	IV
表目录	V
1 实验目的	1
2 实验原理	2
2.1 时域采样定理	2
2.2 频域采样定理	3
3 实验内容与结果分析	4
3.1 主程序框图	4
3.2 时域采样理论验证	4
3.3 频域采样理论验证	7
4 实验结论	9
5 思考题	10
5.1 当 $N < M$ 时如何用一次最少点数的 DFT 得到频谱采样	10
附录 A 程序清单	11

插图目录

图 3.1 实验一主程序框图	4
图 3.2 不同采样频率下的采样序列和 64 点 FFT 幅频特性	6
图 3.3 频域采样及其 IFFT 结果	7

表格目录

表 3.1 三种采样频率对应的采样点数	5
---------------------------	---

1 实验目的

本实验围绕时域采样和频域采样两个基本理论展开。通过对模拟信号进行不同采样频率下的离散化和 FFT 分析，观察采样前后频谱的变化，理解采样频率不足时产生频谱混叠的原因。通过对有限长序列频谱进行 32 点和 16 点等间隔采样，并分别作 IFFT，验证频域采样会导致时域序列按采样点数进行周期延拓；当频域采样点数小于原序列长度时，时域会发生混叠失真。

2 实验原理

2.1 时域采样定理

设模拟信号为 $x_a(t)$ ，以采样周期 T 进行理想采样，采样角频率为

$$\Omega_s = \frac{2\pi}{T}. \quad (2.1)$$

理想采样信号可写为

$$\hat{x}_a(t) = \sum_{n=-\infty}^{\infty} x_a(nT)\delta(t - nT). \quad (2.2)$$

其频谱为原模拟信号频谱的周期延拓：

$$\hat{X}_a(j\Omega) = \frac{1}{T} \sum_{m=-\infty}^{\infty} X_a[j(\Omega - m\Omega_s)]. \quad (2.3)$$

因此，时域采样会使频域频谱以 Ω_s 为周期重复。当采样角频率满足

$$\Omega_s \geq 2\Omega_m \quad (2.4)$$

时，采样后频谱不发生混叠，其中 Ω_m 为信号最高角频率。若采样频率不足，则周期延拓的频谱相互重叠，产生频谱混叠失真。

在计算机实验中，模拟信号经采样得到离散序列

$$x(n) = x_a(nT). \quad (2.5)$$

序列的 DTFT 与理想采样信号频谱之间满足变量替换关系

$$\omega = \Omega T, \quad (2.6)$$

因此可以用 FFT 近似观察采样序列的幅频特性。

2.2 频域采样定理

对离散序列 $x(n)$ 的频谱 $X(e^{j\omega})$ 在 $[0, 2\pi]$ 上等间隔采样 N 点：

$$X_N(k) = X\left(e^{j\frac{2\pi k}{N}}\right), \quad k = 0, 1, \dots, N-1. \quad (2.7)$$

对 $X_N(k)$ 作 N 点 IDFT，得到

$$x_N(n) = \text{IDFT}\{X_N(k)\} = \sum_{r=-\infty}^{\infty} x(n + rN), \quad n = 0, 1, \dots, N-1. \quad (2.8)$$

由此可知，频域采样会导致时域序列按 N 为周期进行延拓并在主值区相加。若原序列长度为 M ，当 $N \geq M$ 时不会发生时域混叠；当 $N < M$ 时，不同周期的样值会叠加到同一主值区位置，产生时域混叠。

时域采样与频域采样具有对偶性：时域采样导致频域周期延拓，频域采样导致时域周期延拓。

3 实验内容与结果分析

3.1 主程序框图

实验主程序流程如图 3.1 所示。程序首先设置采样参数并生成三组采样序列，然后用 64 点 FFT 观察不同采样频率下的幅频特性；接着构造有限长三角序列，分别进行 32 点和 16 点频域采样，并通过 IFFT 验证频域采样引起的时域周期延拓。

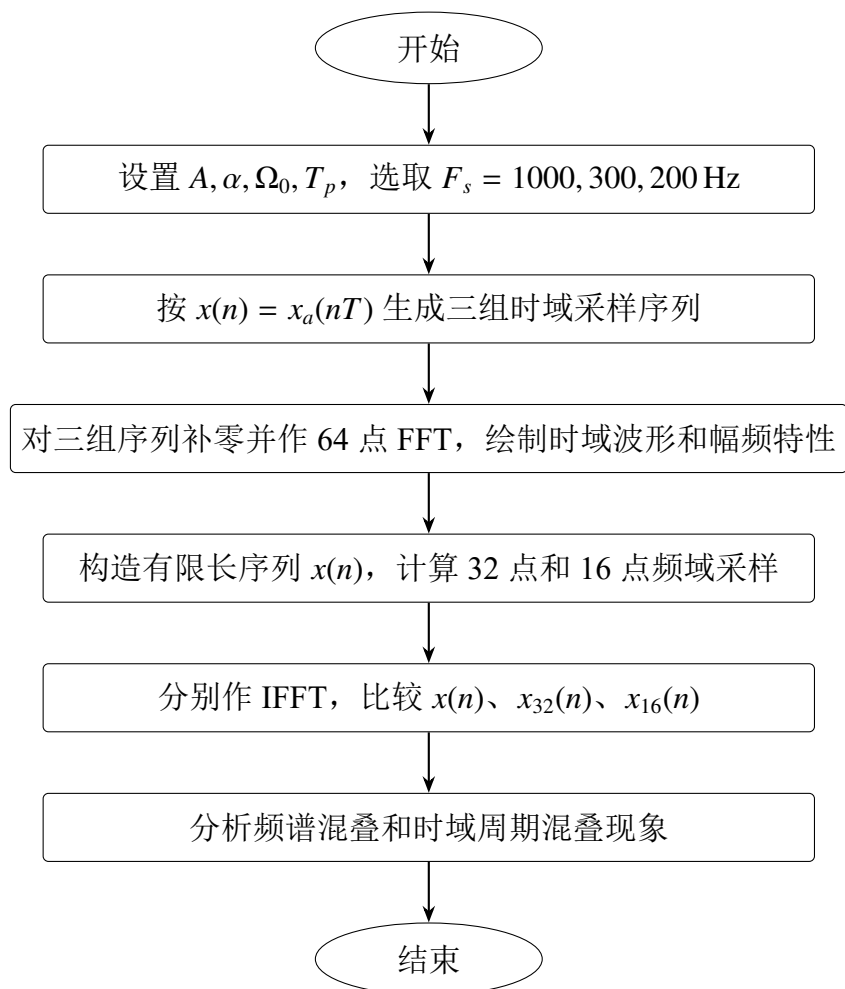


图 3.1 实验一主程序框图

3.2 时域采样理论验证

实验给定模拟信号为

$$x_a(t) = Ae^{-\alpha t} \sin(\Omega_0 t)u(t), \quad (3.1)$$

其中

$$A = 444.128, \quad \alpha = 50\sqrt{2}\pi, \quad \Omega_0 = 50\sqrt{2}\pi \text{ rad/s}. \quad (3.2)$$

取观测时间 $T_p = 50 \text{ ms}$ ，分别采用三种采样频率

$$F_s = 1000 \text{ Hz}, \quad 300 \text{ Hz}, \quad 200 \text{ Hz}.$$

采样序列长度按

$$N = T_p F_s \quad (3.3)$$

计算，得到表 3.1 所示结果。FFT 点数统一取 $M = 64$ ，长度不足 64 的序列在尾部补零。

表 3.1 三种采样频率对应的采样点数

采样频率 F_s	采样点数 N	奈奎斯特频率 $F_s/2$
1000 Hz	50	500 Hz
300 Hz	15	150 Hz
200 Hz	10	100 Hz

实验曲线如图 3.2 所示。

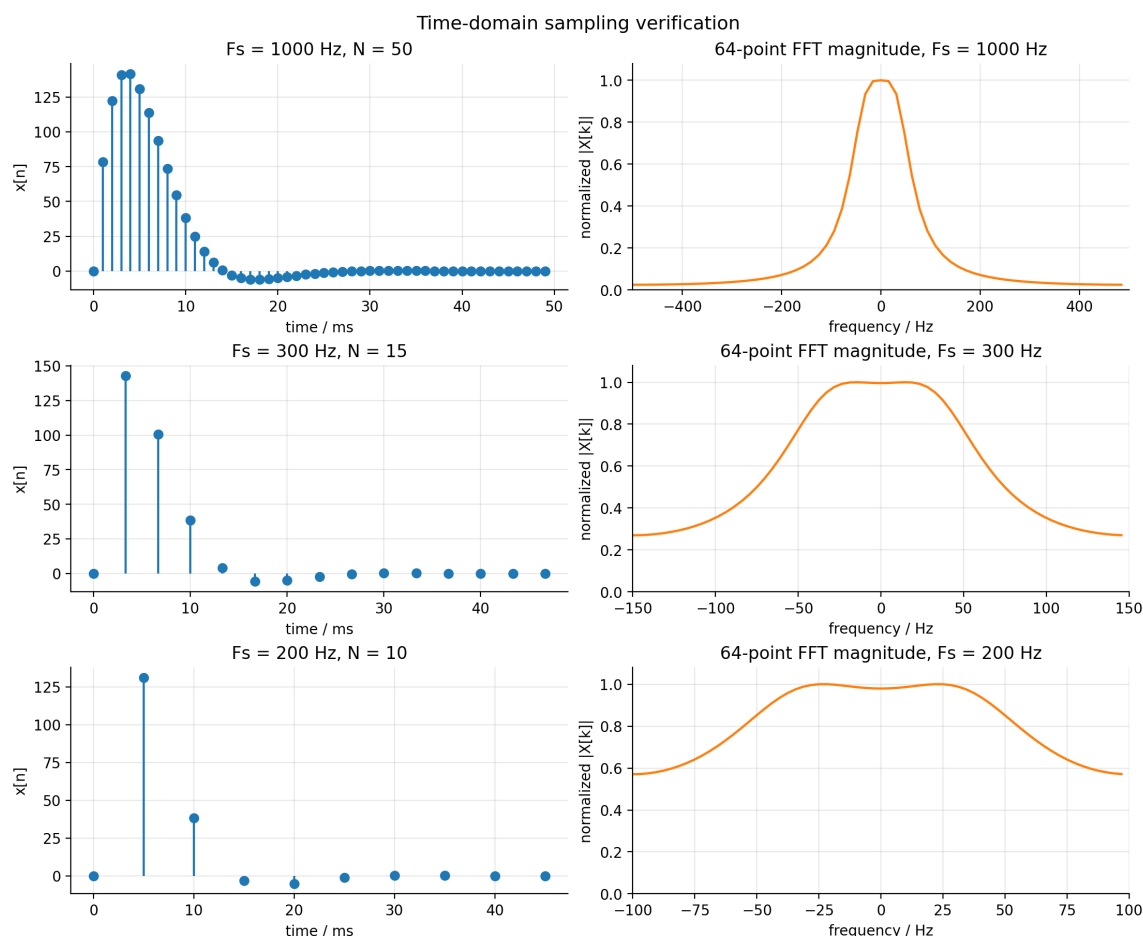


图 3.2 不同采样频率下的采样序列和 64 点 FFT 幅频特性

从图中可以看出，当 $F_s = 1000\text{ Hz}$ 时，采样点较密，奈奎斯特频率较高，采样后频谱在主频带内形状集中，未出现明显混叠。当 $F_s = 300\text{ Hz}$ 时，采样点数减少，频谱曲线明显变宽，频谱两侧的周期延拓成分开始对主频带产生影响，出现轻微混叠趋势。当 $F_s = 200\text{ Hz}$ 时，采样点数只有 10 点，奈奎斯特频率降低到 100 Hz ，频谱周期延拓之间重叠明显，幅频特性发生较大变形，说明采样频率不足会造成频谱混叠。

由于该指数衰减正弦信号并不是严格带限信号，其频谱理论上有无限延伸的尾部，所以实际实验中应结合主要能量范围判断混叠程度。采样频率越高，周期延拓频谱之间的间隔越大，混叠越弱；采样频率越低，频谱重叠越明显，恢复原信号越困难。

3.3 频域采样理论验证

实验给定有限长序列

$$x(n) = \begin{cases} n+1, & 0 \leq n \leq 13, \\ 27-n, & 14 \leq n \leq 26, \\ 0, & \text{其他.} \end{cases} \quad (3.4)$$

该序列长度为 $M = 27$ 。分别对 $X(e^{j\omega})$ 在 $[0, 2\pi]$ 上采样 32 点和 16 点，得到 $X_{32}(k)$ 和 $X_{16}(k)$ ，再分别作 IFFT 得到 $x_{32}(n)$ 和 $x_{16}(n)$ 。实验曲线如图 3.3 所示。

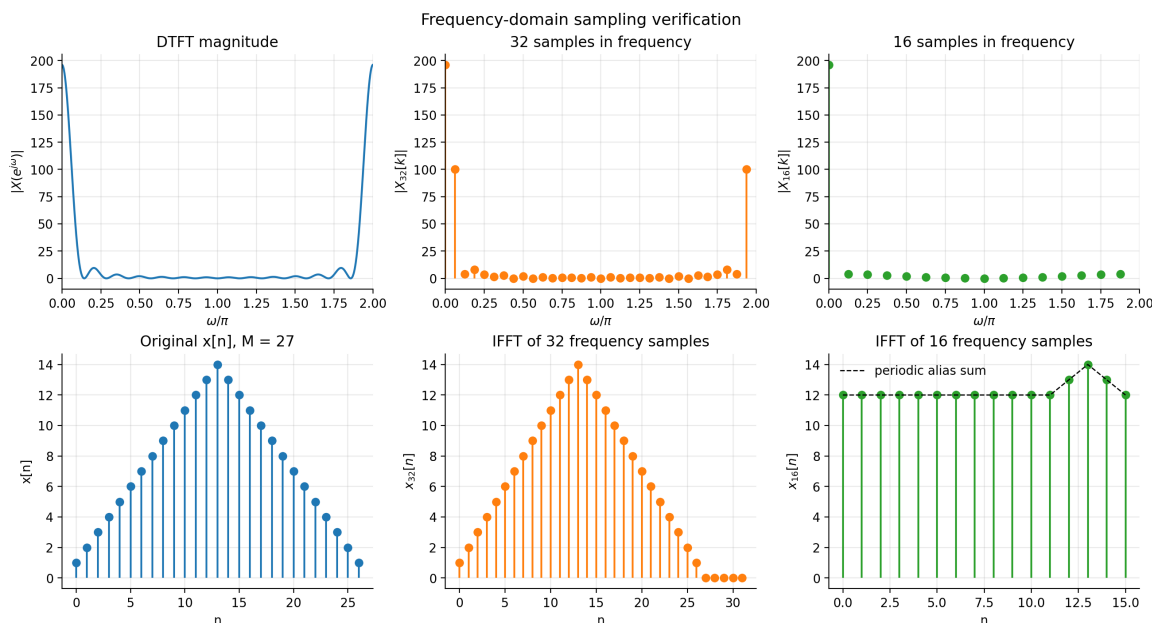


图 3.3 频域采样及其 IFFT 结果

当 $N = 32$ 时， $N > M$ ，因此 32 点 IFFT 得到的 $x_{32}(n)$ 与原序列 $x(n)$ 相同，只是在尾部补了 $32 - 27 = 5$ 个零点，没有发生时域混叠。

当 $N = 16$ 时， $N < M$ ，频域采样点数小于原序列长度。16 点 IFFT 得到的序列不是原序列的前 16 点，而是原序列按 16 为周期延拓后的主值区叠加结果：

$$x_{16}(n) = \sum_{r=-\infty}^{\infty} x(n + 16r), \quad n = 0, 1, \dots, 15. \quad (3.5)$$

计算得到

$$x_{16}(n) = [12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 13, 14, 13, 12].$$

该结果与 16 点 IFFT 的输出一致，说明频域采样点数不足会在时域产生周期混叠。

4 实验结论

本实验分别验证了时域采样定理和频域采样定理。时域采样会使模拟信号频谱按采样频率周期延拓，采样频率越高，频谱副本之间间隔越大，越不容易发生混叠；采样频率过低时，频谱副本相互重叠，采样后信号无法完整表示原信号。使用 FFT 分析采样序列时，FFT 点数决定频域显示的离散频率刻度，补零可以使频谱曲线显示更平滑，但不能增加原采样数据中实际包含的信息。

频域采样会使时域序列周期延拓。若频域采样点数 $N \geq M$ ，IDFT 结果能保持原序列；若 $N < M$ ，IDFT 结果为原序列按 N 折叠后的叠加序列，会发生时域混叠。两个采样定理体现了傅里叶变换中的对偶关系：一个域中的离散采样，会导致另一个域中的周期延拓。

5 思考题

5.1 当 $N < M$ 时如何用一次最少点数的 DFT 得到频谱采样

如果序列 $x(n)$ 的长度为 M ，希望得到其频谱 $X(e^{j\omega})$ 在 $[0, 2\pi]$ 上的 N 点等间隔采样，当 $N < M$ 时，不能简单取原序列的前 N 点作 N 点 DFT，因为这样会丢掉后面的样值。

正确方法是先将原序列按模 N 折叠相加，得到长度为 N 的周期混叠序列：

$$x_N(n) = \sum_{r=-\infty}^{\infty} x(n + rN), \quad n = 0, 1, \dots, N-1. \quad (5.1)$$

然后对 $x_N(n)$ 作一次 N 点 DFT：

$$X_N(k) = \sum_{n=0}^{N-1} x_N(n) e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1. \quad (5.2)$$

即可得到所需的频谱采样值

$$X_N(k) = X\left(e^{j\frac{2\pi k}{N}}\right). \quad (5.3)$$

这里的一次 N 点 DFT 就是最少点数的 DFT，因为目标本身需要得到 N 个等间隔频域采样值。

附录 A 程序清单

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pathlib import Path
4
5
6 OUT_DIR = Path("figures")
7 OUT_DIR.mkdir(exist_ok=True)
8
9
10 def set_common_style():
11     plt.rcParams.update(
12         {
13             "figure.dpi": 120,
14             "savefig.dpi": 200,
15             "axes.grid": True,
16             "grid.alpha": 0.28,
17             "axes.spines.top": False,
18             "axes.spines.right": False,
19             "font.size": 10,
20         }
21     )
22
23
24 def analog_signal_samples(fs, tp=0.05):
25     a = 444.128
26     alpha = 50 * np.sqrt(2) * np.pi
27     omega0 = 50 * np.sqrt(2) * np.pi
28     n = np.arange(int(round(tp * fs)))
29     t = n / fs
30     x = a * np.exp(-alpha * t) * np.sin(omega0 * t)
31     return n, t, x
```

```
32
33
34 def plot_time_domain_sampling():
35     fft_len = 64
36     sample_rates = [1000, 300, 200]
37     fig, axes = plt.subplots(3, 2, figsize=(11, 9),
38                             constrained_layout=True)
39
40     for row, fs in enumerate(sample_rates):
41         n, t, x = analog_signal_samples(fs)
42         padded = np.zeros(fft_len)
43         padded[: len(x)] = x
44         spectrum = np.fft.fftshift(np.fft.fft(padded, fft_len))
45         freq = np.fft.fftshift(np.fft.fftfreq(fft_len, d=1 / fs))
46         mag = np.abs(spectrum)
47         if mag.max() > 0:
48             mag = mag / mag.max()
49
50         ax_time = axes[row, 0]
51         ax_freq = axes[row, 1]
52
53         ax_time.stem(t * 1000, x, basefmt=" ", linefmt="C0-",
54                     markerfmt="C0o")
55         ax_time.set_title(f"Fs = {fs} Hz, N = {len(x)}")
56         ax_time.set_xlabel("time / ms")
57         ax_time.set_ylabel("x[n]")
58
59         ax_freq.plot(freq, mag, color="C1", linewidth=1.6)
60         ax_freq.set_title(f"64-point FFT magnitude, Fs = {fs} Hz")
61         ax_freq.set_xlabel("frequency / Hz")
62         ax_freq.set_ylabel("normalized |X[k]|")
63         ax_freq.set_xlim(-fs / 2, fs / 2)
64         ax_freq.set_ylim(0, 1.08)
65
66     fig.suptitle("Time-domain sampling verification", fontsize=13)
```

```
65     fig.savefig(OUT_DIR / "time_domain_sampling.png", bbox_inches="
        tight")
66     plt.close(fig)
67
68
69 def build_frequency_domain_sequence():
70     n = np.arange(27)
71     x = np.zeros_like(n, dtype=float)
72     x[(0 <= n) & (n <= 13)] = n[(0 <= n) & (n <= 13)] + 1
73     x[(14 <= n) & (n <= 26)] = 27 - n[(14 <= n) & (n <= 26)]
74     return n, x
75
76
77 def periodize_to_length(x, period):
78     folded = np.zeros(period, dtype=float)
79     for i, value in enumerate(x):
80         folded[i % period] += value
81     return folded
82
83
84 def plot_frequency_domain_sampling():
85     n, x = build_frequency_domain_sequence()
86
87     x32 = np.zeros(32)
88     x32[: len(x)] = x
89     x16_alias = periodize_to_length(x, 16)
90
91     x32_spec = np.fft.fft(x32, 32)
92     x16_spec = x32_spec[::2]
93     x32_ifft = np.fft.ifft(x32_spec).real
94     x16_ifft = np.fft.ifft(x16_spec).real
95
96     dense_n = np.arange(len(x))
97     omega = np.linspace(0, 2 * np.pi, 1024, endpoint=False)
98     dense_spec = np.array(
```

```
99         [np.sum(x * np.exp(-1j * w * dense_n)) for w in omega]
100     )
101
102     fig, axes = plt.subplots(2, 3, figsize=(13, 7),
103                             constrained_layout=True)
104
105     axes[0, 0].plot(omega / np.pi, np.abs(dense_spec), color="C0",
106                    linewidth=1.6)
107     axes[0, 0].set_title("DTFT magnitude")
108     axes[0, 0].set_xlabel(r" $\omega/\pi$ ")
109     axes[0, 0].set_ylabel(r" $|X(e^{j\omega})|$ ")
110     axes[0, 0].set_xlim(0, 2)
111
112     k32 = np.arange(32)
113     axes[0, 1].stem(
114         2 * k32 / 32,
115         np.abs(x32_spec),
116         basefmt=" ",
117         linefmt="C1-",
118         markerfmt="C1o",
119     )
120     axes[0, 1].set_title("32 samples in frequency")
121     axes[0, 1].set_xlabel(r" $\omega/\pi$ ")
122     axes[0, 1].set_ylabel(r" $|X_{32}[k]|$ ")
123     axes[0, 1].set_xlim(0, 2)
124
125     k16 = np.arange(16)
126     axes[0, 2].stem(
127         2 * k16 / 16,
128         np.abs(x16_spec),
129         basefmt=" ",
130         linefmt="C2-",
131         markerfmt="C2o",
132     )
133     axes[0, 2].set_title("16 samples in frequency")
```

```
132 axes[0, 2].set_xlabel(r" $\omega/\pi$ ")
133 axes[0, 2].set_ylabel(r" $|X_{16}[k]|$ ")
134 axes[0, 2].set_xlim(0, 2)
135
136 axes[1, 0].stem(n, x, basefmt=" ", linefmt="C0-", markerfmt="C0o
    ")
137 axes[1, 0].set_title("Original x[n], M = 27")
138 axes[1, 0].set_xlabel("n")
139 axes[1, 0].set_ylabel("x[n]")
140
141 axes[1, 1].stem(
142     np.arange(32),
143     x32_ifft,
144     basefmt=" ",
145     linefmt="C1-",
146     markerfmt="C1o",
147 )
148 axes[1, 1].set_title("IFFT of 32 frequency samples")
149 axes[1, 1].set_xlabel("n")
150 axes[1, 1].set_ylabel(r" $x_{32}[n]$ ")
151
152 axes[1, 2].stem(
153     np.arange(16),
154     x16_ifft,
155     basefmt=" ",
156     linefmt="C2-",
157     markerfmt="C2o",
158 )
159 axes[1, 2].plot(
160     np.arange(16),
161     x16_alias,
162     "k--",
163     linewidth=1,
164     label="periodic alias sum",
165 )
```

```
166     axes[1, 2].set_title("IFFT of 16 frequency samples")
167     axes[1, 2].set_xlabel("n")
168     axes[1, 2].set_ylabel(r" $x_{16}[n]$ ")
169     axes[1, 2].legend(frameon=False)
170
171     fig.suptitle("Frequency-domain sampling verification", fontsize
172                 =13)
173     fig.savefig(OUT_DIR / "frequency_domain_sampling.png",
174                 bbox_inches="tight")
175     plt.close(fig)
176
177     return x16_alias, x16_ifft
178
179 def main():
180     set_common_style()
181     plot_time_domain_sampling()
182     x16_alias, x16_ifft = plot_frequency_domain_sampling()
183     print("Generated figures:")
184     print(f"- {OUT_DIR / 'time_domain_sampling.png'}")
185     print(f"- {OUT_DIR / 'frequency_domain_sampling.png'}")
186     print("x16 periodic alias sum:")
187     print(np.round(x16_alias, 6))
188     print("x16 from IFFT:")
189     print(np.round(x16_ifft, 6))
190
191 if __name__ == "__main__":
192     main()
```

代码 A.1 实验一 Python 程序